

Complete AWS Deployment — Empty Account to Running App

Standalone. Nothing assumed, nothing referenced elsewhere. Start with an empty AWS account, finish with the application live on HTTPS.

Every command in order. Every value explained — no `<PLACEHOLDER>` left dangling.

Total time: ~90 minutes, most of it waiting on AWS. **Cost:** ~\$5-8/day while running. Phase 12 tears it all down.

Phase map

Phase	What	Time
0	Prerequisites + AWS account	15 min
1	Fix the code (3 remaining changes)	10 min
2	Cluster + networking	20 min
3	Cluster add-ons	10 min
4	ECR + build + push images	15 min
5	TLS certificate + DNS	10 min
6	Secrets — sealed, not plaintext	10 min
7	Deploy the application	10 min
8	Expose via ALB	5 min
9	Lock the network down	5 min
10	Observability — and prove it collects	10 min
11	Verify, including the security checks	10 min
12	Day-2 operations	—
13	Cost control	—
14	Teardown	15 min

Phases 9 and 10 are not optional extras. Applying default-deny after the app is running is deliberate — it is the order that catches what the policy breaks — and the monitoring phase ends by proving the targets are actually **up**, because this stack once sat with all ten of them down and nobody noticed.

Set these once and keep the terminal open — every phase uses them:

```
export AWS_REGION=ap-south-1
export CLUSTER=ecommerce-cluster
export DOMAIN=yourdomain.com # a domain you actually own
export PROJECT=$HOME/Desktop/ecommerce
cd $PROJECT
```

PHASE 0 — Prerequisites

0.1 Install the tools

```
# Java 17
sudo apt install -y openjdk-17-jdk maven git jq unzip

# AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install && rm -rf aws awscliv2.zip

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl && rm kubectl

# eksctl
```

```
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz" \  
| tar xz -C /tmp && sudo mv /tmp/eksctl /usr/local/bin
```

helm

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

docker

```
sudo apt install -y docker.io
```

```
sudo usermod -aG docker $USER      # log out and back in after this
```

Verify all of them:

```
java -version && mvn -v && aws --version && kubectl version --client \  
&& eksctl version && helm version --short && docker --version
```

0.2 AWS credentials

In the AWS Console: **IAM** → **Users** → **Create user** → attach `AdministratorAccess` → **Security credentials** → **Create access key** → **CLI**.

```
aws configure
# AWS Access Key ID: AKIA...
# AWS Secret Access Key: ...
# Default region name: ap-south-1
# Default output format: json
```

Confirm it works — this must succeed before continuing:

```
aws sts get-caller-identity

{ "UserId": "AIDA...", "Account": "123456789012", "Arn": "arn:aws:iam::123456789012:user/you" }
```

Save your account ID — several later commands need it:

```
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
echo $REGISTRY
```

0.3 A domain you own

Required for HTTPS. `.local` is **not** a real TLD — ACM can never issue a certificate for it, and the HTTPS listener will fail.

- Already own one elsewhere? Fine — you'll point its nameservers at Route 53 in Phase 5.
 - Don't own one? **Route 53 → Registered domains → Register** (~\$12/year, `.click` and `.link` are cheapest).
-

PHASE 1 — Fix the code

The repository still will not deploy to AWS as-is, but the list is shorter than it was. Four of the seven changes this guide used to describe are now in the repository:

Was	Now
Kong Service → ClusterIP	✅ done — the Service carried <code>aws-load-balancer-type: "alb"</code> , which is not a value the annotation recognises and would have provisioned a second, Classic load balancer beside the ALB
Stop the deploy script editing tracked files	✅ done — the <code>sed -i</code> is gone
Remove the manual target registration	✅ done — replaced by <code>k8s/alb/targetgroupbinding.yaml</code> , so the ALB follows pod restarts instead of holding a snapshot
CPU requests so HPA works	✅ done

Three remain. Do them now — finding them mid-deploy wastes an hour.

1.1 Registry URL

`scripts/03-ecr.sh` line 20 hardcodes a local registry:

```
sed -i 's|^REGISTRY=localhost:5100|REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com|' scripts/03-ecr.sh
```

Also add the account lookup just above it:

```
sed -i '/^REGISTRY=/i ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)' scripts/03-ecr.sh
```

Why: `localhost:5100` inside a pod means *that node*, which runs no registry. Every service would sit in `ImagePullBackOff`.

1.2 StorageClass on every volume claim

EKS ships **no default StorageClass**. The 7 `volumeClaimTemplates` (5 Postgres, Redis, Kafka) would stay `Pending` forever and the deploy would abort.

```
sed -i '/accessModes:/i\      storageClassName: gp3' \  
k8s/postgres/postgres.yaml k8s/redis/redis.yaml k8s/kafka/kafka.yaml
```

```
grep -A2 volumeClaimTemplates k8s/postgres/postgres.yaml | head -8
```

You want each to read:

```
volumeClaimTemplates:  
- metadata:  
  name: data  
  spec:  
    storageClassName: gp3  
    accessModes: [ReadWriteOnce]
```

1.3 Remove the local AWS endpoint override

`k8s/services/configmap.yaml` points the AWS SDK at a local emulator that won't exist in the cluster. `notification-service` uses it for SES.

```
sed -i '/AWS_ENDPOINT_URL/d' k8s/services/configmap.yaml
grep -A8 "^data:" k8s/services/configmap.yaml
```

Should now be:

```
data:
  KAFKA_BOOTSTRAP_SERVERS: "kafka-0.kafka.ecommerce.svc.cluster.local:9092"
  REDIS_HOST: "redis-0.redis.ecommerce.svc.cluster.local"
  REDIS_PORT: "6379"
  AWS_DEFAULT_REGION: "ap-south-1"
  SPRING_PROFILE: "prod"
```

1.4 Commit

```
git checkout -b aws-deploy
git add -A && git commit -m "chore: prepare for AWS deployment"
```

PHASE 2 — Cluster and networking

`eksctl` builds the VPC, subnets, IGW, NAT, route tables, IAM roles, node groups and — importantly — **applies the `kubernetes.io/role/*` subnet tags automatically**. Doing this by hand with `aws ec2` is ~25 commands and the tags are the most commonly missed step.

2.1 Cluster config

```
cat > cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: $CLUSTER
  region: $AWS_REGION
  version: "1.29"

vpc:
  cidr: 10.0.0.0/16
  nat:
    gateway: HighlyAvailable # one NAT per AZ – survives an AZ failure
  clusterEndpoints:
    publicAccess: true
    privateAccess: true

iam:
  withOIDC: true # required for IRSA (add-ons in Phase 3)

managedNodeGroups:
  - name: general
    instanceType: t3.large
    minSize: 2
    maxSize: 6
    desiredCapacity: 3
    volumeSize: 30
    privateNetworking: true # nodes get no public IP
    labels: { role: general }
    iam:
      withAddonPolicies:
        ebs: true
        cloudWatch: true
        autoScaler: true

  - name: high-mem
    instanceType: r5.large
    minSize: 1
    maxSize: 4
    desiredCapacity: 2
    volumeSize: 40
    privateNetworking: true
    labels: { role: high-memory }
    iam:
      withAddonPolicies:
        ebs: true
        cloudWatch: true

cloudWatch:
  clusterLogging:
    enableTypes: ["api", "audit", "authenticator", "controllerManager", "scheduler"]
EOF
```

`nat.gateway: HighlyAvailable` creates one NAT per AZ. `Single` is ~\$32/month cheaper but means an AZ-a failure takes internet away from AZ-b as well — which defeats the point of spreading nodes across AZs.

2.2 Create it

```
eksctl create cluster -f cluster.yaml
```

🕒 **15-20 minutes.** It prints progress; leave it alone.

2.3 Verify

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-x-x.ap-south-1.compute.internal	Ready	<none>	2m	v1.29.x
... (5 nodes: 3 general + 2 high-mem)				

```
kubectl get nodes -L role # confirm the labels applied
```

```
aws eks describe-cluster --name $CLUSTER --query 'cluster.status' # "ACTIVE"
```

Confirm the subnet tags exist (this is what the ALB controller needs):

```
aws ec2 describe-subnets \
  --filters "Name=tag:kubernetes.io/cluster/$CLUSTER,Values=shared" \
  --query 'Subnets[].[ID:SubnetId,Tags:Tags[?starts_with(Key,`kubernetes.io/role`)].Key]' \
  --output table
```

You should see `kubernetes.io/role/elb` on public subnets and `kubernetes.io/role/internal-elb` on private ones.

PHASE 3 — Cluster add-ons

Three add-ons. Without them: no storage, no autoscaling, no load balancer.

3.1 EBS CSI driver — storage

```
eksctl create iamserviceaccount \
  --name ebs-csi-controller-sa --namespace kube-system \
  --cluster $CLUSTER --region $AWS_REGION \
  --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
  --approve --role-only --role-name AmazonEKS_EBS_CSI_DriverRole

eksctl create addon --name aws-ebs-csi-driver \
  --cluster $CLUSTER --region $AWS_REGION \
  --service-account-role-arn arn:aws:iam::$ACCOUNT_ID:role/AmazonEKS_EBS_CSI_DriverRole \
  --force
```

The `gp3` `StorageClass` (referenced in Phase 1.2):

```
kubectl apply -f - <<'EOF'
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  type: gp3
  encrypted: "true"
EOF
```

```
kubectl get storageclass
```

`WaitForFirstConsumer` **matters**. It delays creating the EBS volume until the pod is scheduled, so the volume lands in the **same AZ** as the pod. Without it you get volumes in AZ-a and pods in AZ-b that can never attach.

3.2 metrics-server — for HPA

```
kubect1 apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubect1 -n kube-system rollout status deploy/metrics-server --timeout=120s
kubect1 top nodes          # must return numbers, not an error
```

EKS does not ship this. Without it every HPA reads `<unknown>/70%` and silently never scales — no error, no event.

3.3 AWS Load Balancer Controller

This is what turns an Ingress object into a real ALB, and **keeps the target list in sync** as pods restart and scale.

```
# permissions
curl -sO https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.7.1/docs/install/iam_policy.json
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file:///iam_policy.json 2>/dev/null || echo "policy already exists"

# identity — links a K8s ServiceAccount to an IAM role (IRSA)
eksctl create iamserviceaccount \
  --cluster $CLUSTER --region $AWS_REGION \
  --namespace kube-system --name aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn arn:aws:iam::$ACCOUNT_ID:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve

# the controller
helm repo add eks https://aws.github.io/eks-charts && helm repo update
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=$CLUSTER \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=$AWS_REGION

kubectl -n kube-system rollout status deploy/aws-load-balancer-controller --timeout=120s
```

Verify:

```
kubectl -n kube-system logs deploy/aws-load-balancer-controller --tail=20
```

No AccessDenied, no failed to get VPC ID.

IRSA in one line: the pod gets a signed token from Kubernetes, trades it with AWS STS for temporary credentials, and they auto-rotate hourly. No AWS keys stored anywhere.

PHASE 4 — Build and push images

4.1 Create the repositories

```
for SVC in user-service product-service order-service \
  inventory-service payment-service notification-service; do
  aws ecr create-repository --repository-name ecommerce/$SVC \
    --region $AWS_REGION \
    --image-scanning-configuration scanOnPush=true \
    --encryption-configuration encryptionType=AES256 2>/dev/null \
    && echo "created ecommerce/$SVC" || echo "exists ecommerce/$SVC"
done
```

Lifecycle policy — keep only the last 10 images per repo (otherwise storage grows forever):

```
for SVC in user-service product-service order-service \
  inventory-service payment-service notification-service; do
  aws ecr put-lifecycle-policy --repository-name ecommerce/$SVC --region $AWS_REGION \
    --lifecycle-policy-text '{"rules":[{"rulePriority":1,"description":"Keep 10","selection":{"tagStatus":"any","countType":"imageCountMoreT'
done
```

4.2 Build and push

```
export TAG=$(git rev-parse --short HEAD)
echo "Deploying tag: $TAG"

aws ecr get-login-password --region $AWS_REGION \
| docker login --username AWS --password-stdin $REGISTRY

mvn -B -T 1C package -DskipTests

for SVC in user-service product-service order-service \
        inventory-service payment-service notification-service; do
    docker build -t $REGISTRY/ecommerce/$SVC:$TAG $SVC/
    docker push $REGISTRY/ecommerce/$SVC:$TAG
    echo "✅ $SVC:$TAG"
done
```

🕒 ~10-15 minutes on a first run.

Verify:

```
aws ecr describe-images --repository-name ecommerce/order-service \
--region $AWS_REGION --query 'imageDetails[].imageTags' --output text
```

Tag by git SHA, never `latest`. `latest` means something different tomorrow, so there is nothing to roll back to. The SHA ties a running container to an exact commit.

PHASE 5 — TLS certificate and DNS

5.1 Route 53 hosted zone

```
aws route53 create-hosted-zone --name $DOMAIN \
--caller-reference $(date +%s) \
--query 'HostedZone.Id' --output text

export ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name $DOMAIN \
--query 'HostedZones[0].Id' --output text | cut -d/ -f3)
echo $ZONE_ID
```

Point your domain at these nameservers (at your registrar, if the domain wasn't registered in Route 53):

```
aws route53 get-hosted-zone --id $ZONE_ID --query 'DelegationSet.NameServers' --output table
```

🕒 DNS delegation can take up to 48 hours, though it's usually under an hour.

5.2 Request the certificate

```
export CERT_ARN=$(aws acm request-certificate \  
  --domain-name $DOMAIN \  
  --subject-alternative-names ".*$DOMAIN" \  
  --validation-method DNS \  
  --region $AWS_REGION \  
  --query CertificateArn --output text)  
echo $CERT_ARN
```


5.3 Validate it

ACM needs a DNS record proving you control the domain:

```
aws acm describe-certificate --certificate-arn $CERT_ARN --region $AWS_REGION \
--query 'Certificate.DomainValidationOptions[0].ResourceRecord'

{ "Name": "_a1b2c3.yourdomain.com.", "Type": "CNAME", "Value": "_x9y8z7.acm-validations.aws." }
```

Create it automatically:

```
REC=$(aws acm describe-certificate --certificate-arn $CERT_ARN --region $AWS_REGION \
--query 'Certificate.DomainValidationOptions[0].ResourceRecord' --output json)

aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch "{
  \"Changes\": [{\"Action\": \"UPSERT\", \"ResourceRecordSet\": {
    \"Name\": $(echo $REC | jq .Name),
    \"Type\": \"CNAME\", \"TTL\": 300,
    \"ResourceRecords\": [{\"Value\": $(echo $REC | jq .Value)}}]}]"
```

Wait for issuance:

```
aws acm wait certificate-validated --certificate-arn $CERT_ARN --region $AWS_REGION
aws acm describe-certificate --certificate-arn $CERT_ARN --region $AWS_REGION \
--query 'Certificate.Status' --output text      # ISSUED
```

🕒 ~5 minutes once the DNS record propagates.

PHASE 6 — Secrets

A Kubernetes `Secret` is base64. That is **encoding, not encryption** — anyone with read access to the namespace, and anyone who ever sees the YAML, has the values. `k8s/services/secrets.yaml` must never be committed with real content.

The repository ships `k8s/security/sealed-secrets.yaml` instead: the same values encrypted with the cluster's public key, which only the controller's private key can open. That file is safe in Git.

```
# once per cluster
kubectl apply -f https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.27.1/controller.yaml
kubectl -n kube-system rollout status deploy/sealed-secrets-controller

# seal YOUR values — the committed file is sealed for a different cluster
kubeseal --controller-namespace kube-system \
--controller-name sealed-secrets-controller \
--format yaml < k8s/services/secrets.yaml \
> k8s/security/sealed-secrets.yaml
```

The sealing key is per-cluster. A SealedSecret sealed against one cluster cannot be decrypted by another, so re-seal after every cluster rebuild. The symptom otherwise is pods stuck without their `app-secrets`.

`05-deploy.sh` prefers the SealedSecret when the CRD is present and falls back to plaintext with a warning when it is not.

6.1 Fill them in

```
cp k8s/services/secrets.example.yaml k8s/services/secrets.yaml
```

Generate real values:

```
export JWT=$(openssl rand -base64 48)      # must be ≥32 chars
export REDIS_PW=$(openssl rand -base64 24)
export DB_PW=$(openssl rand -base64 24)
```

```
sed -i "s|JWT_SECRET: \".*\"|JWT_SECRET: \"$JWT\"|" k8s/services/secrets.yaml
sed -i "s|REDIS_PASSWORD: \".*\"|REDIS_PASSWORD: \"$REDIS_PW\"|" k8s/services/secrets.yaml
sed -i "s|_DB_USER: \".*\"|_DB_USER: \"postgres\"|g" k8s/services/secrets.yaml
sed -i "s|_DB_PASS: \".*\"|_DB_PASS: \"$DB_PW\"|g" k8s/services/secrets.yaml
```

no AWS keys in Secrets – IRSA handles it (6.3)

```
sed -i '/AWS_ACCESS_KEY_ID/d;/AWS_SECRET_ACCESS_KEY/d' k8s/services/secrets.yaml
```

```
grep -c CHANGE_ME k8s/services/secrets.yaml      # must print 0
```

The Redis password must match `redis.yaml` `'s --requirepass :`

```
grep -n "requirepass" k8s/redis/redis.yaml
```

If it's hardcoded there rather than read from the Secret, update it to match.

`secrets.yaml` is gitignored. Never commit it — only `secrets.example.yaml` belongs in the repo.

6.2 Verify SES for notification emails

```
aws ses verify-email-identity --email-address noreply@$DOMAIN --region $AWS_REGION
```

Check your inbox and click the link. **SES starts in sandbox mode** — it will only send to *verified* addresses until you request production access (**SES** → **Account dashboard** → **Request production access**).

6.3 Give notification-service SES access via IRSA

```
eksctl create iamserviceaccount \  
  --name notification-sa --namespace ecommerce \  
  --cluster $CLUSTER --region $AWS_REGION \  
  --attach-policy-arn arn:aws:iam::aws:policy/AmazonSESEFullAccess \  
  --approve
```

This fails if the `ecommerce` namespace doesn't exist yet. Either create it first (`kubect1 create ns ecommerce`) or run this right after Phase 7.1.

Then add to the notification-service Deployment in `k8s/services/deployments.yaml` :

```
spec:  
  template:  
    spec:  
      serviceAccountName: notification-sa
```

PHASE 7 — Deploy the application

Order matters and each step waits for the previous one.

7.1 Namespaces, secrets, config

```
kubectl apply -f k8s/namespace/namespace.yaml
kubectl apply -f k8s/services/secrets.yaml
sed "s|REGISTRY|$REGISTRY|g" k8s/services/configmap.yaml | kubectl apply -f -

kubectl get ns ecommerce monitoring
kubectl -n ecommerce get secret,configmap
```

Why first: pods read Secrets and ConfigMaps **at startup**. Deploy them after and every pod crash-loops.

7.2 Data layer

```
kubectl apply -f k8s/postgres/postgres.yaml
kubectl apply -f k8s/redis/redis.yaml
kubectl apply -f k8s/kafka/kafka.yaml
```

Watch the volumes bind — this is where a missing StorageClass shows up:

```
kubectl -n ecommerce get pvc -w      # all must reach Bound

for a in postgres-user postgres-product postgres-order postgres-inventory postgres-payment redis kafka; do
  kubectl -n ecommerce wait --for=condition=ready pod -l app=$a --timeout=300s && echo "✅ $a"
done
```

🕒 Kafka takes ~60-90s.

Confirm the topics exist — the manifests set `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"`, so a Job creates them explicitly:

```
kubectl -n ecommerce get job kafka-topics-init
kubectl -n ecommerce exec kafka-0 -- kafka-topics --bootstrap-server localhost:9092 --list

inventory.updated
order.cancelled
order.created
payment.processed
product.created
```

Without these five topics the saga cannot run — orders would stay `PENDING` forever.

7.3 Kong gateway

```
kubectl apply -f k8s/kong/kong.yaml
kubectl -n ecommerce wait --for=condition=ready pod -l app=kong --timeout=180s
kubectl -n ecommerce get svc kong-proxy # must be ClusterIP, not LoadBalancer
```

7.4 Microservices

```
sed "s|REGISTRY|$REGISTRY|g; s|IMAGE_TAG|$TAG|g" k8s/services/deployments.yaml \
| kubectl apply -f -
```

```
for SVC in user-service product-service order-service \
    inventory-service payment-service notification-service; do
    kubectl -n ecommerce rollout status deployment/$SVC --timeout=300s
done
```


7.5 Monitoring

```
kubect1 apply -f k8s/monitoring/monitoring.yaml  
kubect1 -n monitoring get pods
```

7.6 Checkpoint

```
kubectl -n ecommerce get pods
```

Expect **14 pods**, all `Running`: 5 postgres · 1 redis · 1 kafka · 1 kong · 6 services

```
kubectl -n ecommerce get pvc # 7 Bound
```

```
kubectl -n ecommerce get hpa # TARGETS shows numbers, not <unknown>
```

PHASE 8 — Expose via ALB

8.1 The Ingress

```
cat > k8s/ingress.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: ecommerce
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/certificate-arn: $CERT_ARN
    alb.ingress.kubernetes.io/healthcheck-path: /status
    alb.ingress.kubernetes.io/healthcheck-port: '8100'
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '15'
    alb.ingress.kubernetes.io/healthy-threshold-count: '2'
    alb.ingress.kubernetes.io/unhealthy-threshold-count: '3'
    alb.ingress.kubernetes.io/target-group-attributes: deregistration_delay.timeout_seconds=30
    alb.ingress.kubernetes.io/load-balancer-attributes: idle_timeout.timeout_seconds=60
spec:
  ingressClassName: alb
  rules:
    - host: $DOMAIN
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kong-proxy
                port:
                  number: 80
EOF
```

```
kubectl apply -f k8s/ingress.yaml
```

```
kubectl -n ecommerce get ingress -w # ADDRESS appears in ~3 min
```

`deregistration_delay=30` is what makes deploys drop zero requests — when a pod terminates the ALB stops sending it *new* traffic but lets in-flight requests finish for 30 seconds.

If ADDRESS stays empty:

```
kubectl -n kube-system logs deploy/aws-load-balancer-controller --tail=50
```

`couldn't auto-discover subnets` means the subnet tags are missing — but eksctl applied them in Phase 2, so verify with the command in 2.3.

8.2 DNS

```
export ALB_DNS=$(kubectl -n ecommerce get ingress ecommerce-ingress \
-o jsonpath='{.status.loadBalancer.ingress[0].hostname}')

export ALB_ZONE=$(aws elbv2 describe-load-balancers --region $AWS_REGION \
--query "LoadBalancers[?DNSName=='$ALB_DNS'].CanonicalHostedZoneId" --output text)

echo "$ALB_DNS / $ALB_ZONE"

aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch "{
  \"Changes\": [{\"Action\": \"UPSERT\", \"ResourceRecordSet\": {
    \"Name\": \"$DOMAIN\", \"Type\": \"A\",
    \"AliasTarget\": {\"HostedZoneId\": \"$ALB_ZONE\", \"DNSName\": \"$ALB_DNS\",
      \"EvaluateTargetHealth\": true}}}]}"

Look the zone ID up, never hardcode it. It's an AWS constant that differs per region, and a wrong value silently breaks the record.

sleep 60 && dig +short $DOMAIN
```

PHASE 9 — Lock the network down

Kubernetes allows every pod to talk to every other pod. Until this phase, any container in the namespace — a compromised service, a debug pod someone left running, a malicious image — can reach every Postgres instance, Redis and Kafka, and can open outbound connections to anywhere.

Applied **after** the application is running, on purpose. Default-deny takes effect the moment it lands, and you want to be watching when it does.

```
kubectl apply -f k8s/security/networkpolicy.yaml
kubectl -n ecommerce get netpol          # expect 17
```

9.1 What it enforces

Each service reaches **only its own database**. `order-service` cannot open a socket to `postgres-user`. `notification-service` reaches Kafka and nothing else. That is the database-per-service boundary made real instead of being a naming convention.

9.2 Verify it — do not assume the CNI enforces anything

```
kubectl -n ecommerce run np-test --image=busybox:1.36 --restart=Never \
--command -- sleep 300
```

every one of these must FAIL

```
kubectl -n ecommerce exec np-test -- timeout 4 nc -z postgres-user 5432
```

```
kubectl -n ecommerce exec np-test -- timeout 4 nc -z redis 6379
```

```
kubectl -n ecommerce exec np-test -- timeout 4 nc -z kafka 9092
```

```
kubectl -n ecommerce exec np-test -- timeout 4 nc -z 1.1.1.1 443
```

```
kubectl -n ecommerce delete pod np-test
```

On EKS this is the step people skip and regret. The AWS VPC CNI does **not** enforce NetworkPolicy unless the network policy agent is enabled:

```
aws eks update-addon --cluster-name $CLUSTER --addon-name vpc-cni \
--configuration-values '{"enableNetworkPolicy":"true"}
```

Without it the policies apply cleanly, `kubectl get netpol` shows all 17, and **nothing is actually blocked**. Run the busybox test above and believe the result, not the object count.

9.3 Then check the application still works

A default-deny rollout that breaks the app is not a fix:

```
bash .github/smoke-test.sh # 29/29
```

This is also where an earlier attempt went wrong. The policy was verified against the application path and passed — and silently cut Prometheus off from every service it scrapes, because only Kong was allowed in. The policy in the repo now allows the monitoring namespace on the six service ports. If you write your own, remember that observability is the first thing default-deny breaks.

PHASE 10 — Observability, and proof that it collects

Deploying Prometheus and Grafana is the easy half. This stack once ran with all ten scrape targets **down** and zero dashboards, and nobody noticed for the life of the cluster — because Grafana itself was `1/1 Running` and answering `"database": "ok"` the entire time. It stores nothing; it asks Prometheus. Up and useless look identical from outside.

```
kubect1 apply -f k8s/monitoring/ # the whole directory, not just monitoring.yaml
```

The directory matters: Prometheus mounts the `prometheus-rules` ConfigMap and Grafana mounts `grafana-dashboards` and `grafana-dashboard-provider`, which live in `alerts.yaml` and `dashboards.yaml`. Applying only `monitoring.yaml` leaves both pods in `ContainerCreating` waiting on a ConfigMap nothing created.

10.1 Prove the targets are up

This is the checkpoint. Do not move on until it passes.

```
kubect1 -n monitoring port-forward svc/prometheus 9090:9090 &
curl -s 'http://localhost:9090/api/v1/targets?state=active' \
| jq -r '.data.activeTargets[] | "\(.health) \(.labels.job) \(.scrapeUrl)'"'
```

Every line must read up . If any says down , read lastError — it will be one of these four, all of which were real here:

Symptom	Cause
404 on /actuator/prometheus	micrometer-registry-prometheus missing, or prometheus absent from management.endpoints.web.exposure.include
all kubernetes_sd_configs jobs empty	Prometheus running as the default ServiceAccount, which cannot list pods
connection refused from the monitoring namespace	NetworkPolicy from Phase 9 does not allow it in
no such host for a Kafka broker	the scrape config assumes three brokers and a JMX exporter that was never deployed

10.2 Prove there is business data, not just machine data

```
for q in orders_created_total orders_pending_oldest_age_seconds \
    inventory_reserve_total kafka_consumergroup_lag; do
    echo -n "$q -> "
    curl -s "http://localhost:9090/api/v1/query?query=$q" | jq '.data.result | length'
done
```

`orders_pending_oldest_age_seconds` is the one that matters. Four orders once sat at PENDING permanently while the pods were healthy, consumer lag was 0, CPU was normal and nothing had thrown an exception. No infrastructure metric can express *customers' orders are disappearing*. That gauge can, and the alert fires at two minutes because a healthy saga finishes in seconds.

10.3 Grafana

```
kubect1 -n monitoring port-forward svc/grafana 3000:3000 &  
# http://localhost:3000 admin / (GF_SECURITY_ADMIN_PASSWORD)
```

The dashboard is provisioned from a ConfigMap, not clicked together in the UI — one built in the UI dies with the pod.

If every panel reads **"No data"** with an error badge while Prometheus clearly has the data, check the datasource uid. Grafana generates one unless you declare it, and a provisioned dashboard that names `uid: prometheus` will match nothing. `k8s/monitoring/monitoring.yaml` declares it for exactly this reason — and a generated uid also differs on a fresh install, so the dashboard would break on the next cluster even if it worked on this one.

10.4 What is still missing

There is no Alertmanager. The nine rules in `alerts.yaml` evaluate and fire into `alertmanagers: static_configs: targets: []`. Nothing reaches a person.

Before calling this production, deploy Alertmanager and route `severity: critical` to a pager and `warning` to chat. A dashboard only works while someone is watching it, and the incident this stack was built around happened at a moment when nobody was.

PHASE 11 — Verify

11.1 Health

```
kubectl -n ecommerce get pods,svc,ingress,hpa
kubectl -n ecommerce get pvc
```

```
# ALB targets healthy?
TG=$(aws elbv2 describe-target-groups --region $AWS_REGION \
  --query "TargetGroups[?contains(TargetGroupName,'ecommerce')].TargetGroupArn" --output text)
aws elbv2 describe-target-health --target-group-arn $TG --region $AWS_REGION \
  --query 'TargetHealthDescriptions[].TargetHealth.State' --output text    # healthy healthy
```

11.2 End to end

```
export KONG_URL=https://$DOMAIN  
bash .github/smoke-test.sh
```

This exercises **16 endpoints** and pushes a real order through the entire saga to **PAID** — order → Kafka → inventory (Redis lock) → Kafka → payment → notification.

11.3 By hand

```
curl -sX POST https://$DOMAIN/api/users/register \
-H 'Content-Type: application/json' \
-d '{"email":"a@b.com","password":"pass1234","name":"Test"}'

TOKEN=$(curl -sX POST https://$DOMAIN/api/users/login \
-H 'Content-Type: application/json' \
-d '{"email":"a@b.com","password":"pass1234"}' | jq -r .token)

curl -sX POST https://$DOMAIN/api/products \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"name":"Laptop","price":50000,"category":"electronics"}'

curl -sX POST https://$DOMAIN/api/inventory/restock \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"productId":1,"quantity":10}'

curl -sX POST https://$DOMAIN/api/orders \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"productId":1,"quantity":2}'

sleep 5
curl -s https://$DOMAIN/api/orders/1 -H "Authorization: Bearer $TOKEN" | jq .status
# "CONFIRMED"
```

Watch the saga while it happens:

```
kubect1 -n ecommerce logs -l app=order-service -f --tail=20
```

11.4 Monitoring

```
kubect1 -n monitoring port-forward svc/grafana 3000:3000 # admin/admin
```

11.5 The security checks — the ones that actually regress

Every request below returned **200** before the services verified tokens for themselves. Five of the six had no security configuration at all: the gateway issued a JWT and nothing ever checked it.

```
API=https://api.$DOMAIN
BODY='{ "shippingAddress": "x", "items": [{"productId": 1, "productName": "p", "quantity": 1, "price": 10}] }'

# all five must be 403
curl -s -o /dev/null -w "orders no token -> %{http_code}"
" -X POST $API/api/orders -H 'Content-Type: application/json' -d "$BODY"
curl -s -o /dev/null -w "orders bad token -> %{http_code}"
" -X POST $API/api/orders -H 'Authorization: Bearer not.a.token' -H 'Content-Type: application/json' -d "$BODY"
curl -s -o /dev/null -w "orders/user/1 -> %{http_code}"
" $API/api/orders/user/1
curl -s -o /dev/null -w "products no token -> %{http_code}"
" -X POST $API/api/products -H 'Content-Type: application/json' -d '{"name": "x", "price": 1, "category": "c"}'
curl -s -o /dev/null -w "restock no token -> %{http_code}"
" -X POST $API/api/inventory/1/restock -H 'Content-Type: application/json' -d '{"quantity": 9999}'
```

The restock one is the expensive one: an anonymous caller could set any product to any quantity.

Ownership. With a valid token of your own, another user's order must not be readable:

```
curl -s -o /dev/null -w "someone else's order -> %{http_code}"
" -H "Authorization: Bearer $MY_TOKEN" $API/api/orders/$THEIR_ORDER_ID
```

Expect **404**, not 403. 403 confirms the id is real, which is all an attacker needs to walk the order table one id at a time.

A userid in the body must be ignored. It used to be obeyed, which let any caller book an order in anyone's name:

```
curl -s -X POST $API/api/orders -H "Authorization: Bearer $MY_TOKEN" -H 'Content-Type: application/json' -d '{"userId": 999999, "shippingA
# must be YOUR id, not 999999
```

`.github/smoke-test.sh` runs all of the above plus the saga, as three separate principals. **29 checks, and the denials are the point** — they are the regression nothing else would notice.

11.6 Does the ALB actually follow the pods

The single most common way this deployment rots: the target group holds the IP of a pod that no longer exists.

```
TG=$(aws elbv2 describe-target-groups --names ecommerce-kong-tg --query 'TargetGroups[0].TargetGroupArn' --output text)
```

```
aws elbv2 describe-target-health --target-group-arn $TG --query 'TargetHealthDescriptions[0].Target.Id' --output text
kubectl -n ecommerce get pods -l app=kong -o jsonpath='{range .items[*]}{.status.podIP}{
"}{end}'
```

The two lists must match. Then kill a Kong pod and watch them match again within about fifteen seconds — that is the `TargetGroupBinding` doing its job. If they drift apart and stay apart, the AWS Load Balancer Controller is not running or its `serviceRef.port` is wrong.

`serviceRef.port` is the **Service** port (80), not the container port (8000). Given the container port the controller reports `Successfully reconciled` while logging `BackendNotFound`, and the target group stays empty. Check the targets, not the status.

PHASE 12 — Day-2 operations

Deploy a new version

```
export TAG=$(git rev-parse --short HEAD)
# build + push (Phase 4.2), then:
kubectl -n ecommerce set image deployment/order-service \
  order-service=$REGISTRY/ecommerce/order-service:$TAG
kubectl -n ecommerce rollout status deployment/order-service
```

Roll back

```
kubectl -n ecommerce rollout undo deployment/order-service
kubectl -n ecommerce rollout history deployment/order-service
```

Database migrations do **not** roll back — Flyway runs forward only. During a rolling deploy old and new pods run **simultaneously** against the same database, so every migration must be backward-compatible.

Scale

```
kubectl -n ecommerce scale deployment/order-service --replicas=5
eksctl scale nodegroup --cluster $CLUSTER --name general --nodes 5
```

Logs

```
kubectl -n ecommerce logs -l app=order-service --tail=100 -f
kubectl -n ecommerce logs <pod> --previous # the crashed instance
```

Inspect state

```
kubectl -n ecommerce exec -it postgres-order-0 -- psql -U postgres -d orderdb \
-c 'SELECT id,status FROM orders ORDER BY id DESC LIMIT 5;'
```

```
kubectl -n ecommerce exec redis-0 -- redis-cli -a "$REDIS_PW" KEYS '*'
```

```
kubectl -n ecommerce exec kafka-0 -- kafka-console-consumer \
--bootstrap-server localhost:9092 --topic order.created --from-beginning --max-messages 5
```

PHASE 13 — Cost control

Roughly **\$5-8/day** with this setup:

Item	~Monthly
EKS control plane	\$73
3× t3.large + 2× r5.large	~\$300
2× NAT Gateway (HA)	~\$65
ALB	~\$20
EBS (7 volumes)	~\$15

Cut it while learning:

```
# scale nodes to zero overnight – control plane still bills
eksctl scale nodegroup --cluster $CLUSTER --name general --nodes 0
eksctl scale nodegroup --cluster $CLUSTER --name high-mem --nodes 0
```

Or use `nat.gateway: Single` in `cluster.yaml` (saves ~\$32/month, at the cost of AZ resilience), and `t3.medium` instead of `t3.large`.

Set a billing alarm now: Console → **Billing** → **Budgets** → **Create budget** → Cost budget → \$20/month → alert at 80%.

PHASE 14 — Teardown

Order matters — Kubernetes-created AWS resources must go before the cluster.

```
# 1. Ingress first – this releases the ALB
kubectl delete -f k8s/ingress.yaml
sleep 60

# 2. The app (this also deletes the EBS volumes)
kubectl delete namespace ecommerce monitoring

# 3. The cluster – eksctl removes VPC, subnets, NAT, IGW, node groups, roles
eksctl delete cluster --name $CLUSTER --region $AWS_REGION --wait
```

🕒 ~15 minutes.

Then clean up what eksctl doesn't own:

```
# ECR repositories
for SVC in user-service product-service order-service \
    inventory-service payment-service notification-service; do
    aws ecr delete-repository --repository-name ecommerce/$SVC \
    --region $AWS_REGION --force
done

# certificate
aws acm delete-certificate --certificate-arn $CERT_ARN --region $AWS_REGION

# hosted zone (delete its records first)
aws route53 delete-hosted-zone --id $ZONE_ID
```

Verify nothing is left billing:

```
aws ec2 describe-nat-gateways --region $AWS_REGION \
    --filter Name=state,Values=available --query 'NatGateways[].NatGatewayId'
aws ec2 describe-addresses --region $AWS_REGION --query 'Addresses[].AllocationId'
aws elbv2 describe-load-balancers --region $AWS_REGION --query 'LoadBalancers[].LoadBalancerName'
aws ec2 describe-volumes --region $AWS_REGION \
    --filters Name=status,Values=available --query 'Volumes[].VolumeId'
```

All four should return empty. **Unattached Elastic IPs and orphaned NAT Gateways are the two that quietly keep charging.**

Troubleshooting

Symptom	Command	Cause
ImagePullBackOff	<code>kubectl -n ecommerce describe pod <p></code>	registry URL (Phase 1.1), or ECR login expired (12h)
PVC Pending	<code>kubectl -n ecommerce describe pvc <p></code>	StorageClass missing (1.2 / 3.1)
PVC Bound, pod won't schedule	<code>kubectl describe pod</code>	volume in wrong AZ — needs <code>WaitForFirstConsumer</code>
Ingress ADDRESS empty	<code>kubectl -n kube-system logs deploy/aws-load-balancer-controller</code>	subnet tags, or controller IAM
ALB up, 503	<code>aws elbv2 describe-target-health --target-group-arn \$TG</code>	Kong not passing <code>/status:8100</code>
CrashLoopBackOff	<code>kubectl logs <pod> --previous</code>	wrong DB credentials, or DB not ready
JWT secret too short	startup logs	<code>JWT_SECRET</code> under 32 chars
Order stuck PENDING	<code>kubectl -n ecommerce logs -l app=inventory-service</code>	Kafka topics missing (7.2)

Symptom	Command	Cause
Redis NOAUTH	any service log	password mismatch between Secret and redis.yaml
HPA <unknown>	kubectl top pods -n ecommerce	metrics-server (3.2) or no CPU requests (1.7)
Emails not arriving	notification logs	SES sandbox, or identity unverified (6.2)
Certificate stuck pending	aws acm describe-certificate ...	validation CNAME missing or DNS not delegated

Full sequence

```
# 0 - setup
export AWS_REGION=ap-south-1 CLUSTER=ecommerce-cluster DOMAIN=yourdomain.com
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

# 1 - code fixes (manual, Phase 1)

# 2 - cluster
eksctl create cluster -f cluster.yaml

# 3 - add-ons
eksctl create addon --name aws-ebs-csi-driver --cluster $CLUSTER --force
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=$CLUSTER --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

# 4 - images
export TAG=$(git rev-parse --short HEAD)
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $REGISTRY
mvn -B -T 1C package -DskipTests
# build + push loop

# 5 - cert + DNS
# 6 - secrets
# 7 - deploy
kubectl apply -f k8s/namespace/ -f k8s/services/secrets.yaml
kubectl apply -f k8s/postgres/ -f k8s/redis/ -f k8s/kafka/ -f k8s/kong/
sed "s|REGISTRY|$REGISTRY|g; s|IMAGE_TAG|$TAG|g" k8s/services/deployments.yaml | kubectl apply -f -

# 8 - expose
kubectl apply -f k8s/ingress.yaml

# 9 - verify
KONG_URL=https://$DOMAIN bash .github/smoke-test.sh

# 12 - teardown
eksctl delete cluster --name $CLUSTER --wait
```